

Seminário de Programação Competitiva

Giovanni Daldegan e Pedro Henrique Braga

1 de julho de 2026

1 Increasing Subsequence

<https://cses.fi/problemset/task/1145>

You are given an array containing n integers. Your task is to determine the longest increasing subsequence in the array, i.e., the longest subsequence where every element is larger than the previous one.

A subsequence is a sequence that can be derived from the array by deleting some elements without changing the order of the remaining elements.

Input

The first line contains an integer n : the size of the array.

After this there are n integers $x_1, x_2, \dots, x_n$: the contents of the array.

Output

Print the length of the longest increasing subsequence.

Constraints

$$1 \leq n \leq 2 \cdot 10^5$$

$$1 \leq x_i \leq 10^9$$

Example

Input:

8

7 3 5 3 6 2 9 8

Output:

4

Observação 1: Subsequência $\neq$ Subarray, enquanto que um subarray é necessário um conjunto contíguo/ininterrupto de elementos do array, uma subsequência basta que os elementos estejam em ordem, não necessariamente contíguos, i.e.: o conjunto $\{3, 5, 6, 9\}$ é, nesta ordem, uma subsequência do array, mas não é um subarray, e o conjunto $\{7, 3, 5, 3\}$ é, nesta ordem, além de uma subsequência um subarray do array original.

Observação 2: Note que neste problema não é necessário retornar a maior subsequência crescente que existe no array, basta retornar seu tamanho, essa "restrição" permite (como veremos a seguir) a construção de uma solução com complexidade inferior se comparado com o problema de retornar além do tamanho, os elementos que compõe essa subsequência.

2 Explicação do Problema

O objetivo principal do problema proposto é: dado um array contendo n números inteiros positivos, identificar nessa estrutura, qual a maior quantidade possível de elementos que eu posso escolher desse array de tal forma que um elemento seja sempre exclusivamente maior que o elemento anterior. Nesse caso dizemos qualquer quantidade de elementos e não uma sequência contínua de elementos do array uma vez que, conforme dito na **observação 1** o problema busca subsequências e não subarrays.

Analisando as restrições do problema e os casos extremos das variáveis envolvidas; os elementos do array são números inteiros que variam entre 1 e 10^9 , então podem ser representados pelo tipo inteiro (int), e o tamanho do array pode variar entre 1 e $2 \cdot 10^5$ e isso evidencia que soluções que performem em um tempo, em notação big-O, de $\mathcal{O}(n^2)$ não serão viáveis uma vez que isso requeriria no pior caso um número de operações da ordem de 10^{10} , o que não pode ser realizado, dentro das restrições propostas em 1 segundo. Desse modo, a solução almejada deve ser programada com complexidade de tempo de $\mathcal{O}(n \log n)$, o que resultaria em um número de operações da ordem de 10^6 , o que é viável em 1 segundo; ou inferior para que a solução passe no juiz online.

Para o caso teste disponibilizado, considera-se um array de tamanho 8 que apresenta os seguintes elementos: $\{7, 3, 5, 3, 6, 2, 9, 8\}$, a partir desse caso de teste as possíveis subsequências crescentes geradas que apresentam o maior comprimento possível são: $\{3, 5, 6, 8\}$ e $\{3, 5, 6, 9\}$ é possível mostrar que ambas as subsequências enunciadas são as que apresentam o maior tamanho, e sendo assim o programa deverá ao final retornar 4, e que não é possível construir uma subsequência que possua um tamanho maior que esse, para este caso de teste considerado.

3 Solução trivial

Uma solução para o problema utilizando técnicas programação dinâmica é a seguinte: para cada elemento do array, vamos obter o tamanho da maior subsequência crescente que termina com ele sendo o último valor. Dessa forma, o elemento que indicar o maior tamanho de LIS (Longest Incremental Subsequence, a maior subsequência crescente) do array nos dá a resposta correta.

Para isso, vamos precisar armazenar dois vetores de mesmo tamanho: **array**, contendo os valores de entrada do programa; e **LIS**, guardando em cada posição i o tamanho máximo de LIS terminada no i -ésimo elemento do array principal. Além disso, precisamos guardar um inteiro com o resultado final da resolução, que será a variável **longest**.

Primeiro, inicializamos o vetor **LIS** com o valor 1 em todas as posições, já que todo elemento forma uma sequência de um único valor. Então, para encontrarmos os valores de **LIS** utilizaremos dois laços **for** aninhados, indexados em i e j , respectivamente, com i percorrendo todas as posições do array e j percorrendo as posições de 0 até $i - 1$.

Assim, podemos receber as entradas do array ao mesmo tempo que computamos qual a maior subsequência crescente terminada nele, pois para cada posição i , já temos as posições

$0 \leq j < i$ anteriores. Se `array[j] < array[i]`, quer dizer que `array[j]` é candidato para a LIS de `array[i]`, pois `array[j]` vem antes e é menor.

Consideremos que a inclusão da LIS de `array[j]` (medida em `LIS[j]`) à LIS de `array[i]` (`LIS[i]`) tem tamanho `LIS[j] + 1`, pois é a LIS até `array[j]` mais o próprio elemento `array[i]`. Caso a LIS de `array[i]` possa crescer com a adição da LIS de `array[j]`, então atualizamos `LIS[i]` para `LIS[j] + 1`. Porém, se a LIS de `array[i]` já é maior com outros elementos e sem `array[j]`, então mantemos `LIS[i]` sem alteração. Esses dois casos são contemplados pelo uso da função `max(LIS[i], LIS[j]+1)`.

Por fim, a cada atualização no vetor `LIS` fazemos uma checagem para ver se `longest` deve ser atualizada para um novo valor máximo. Depois de todas as iterações, o programa mostra a saída correta.

Porém, essa solução tem complexidade $\mathcal{O}(n^2)$, o que extrapola o limite de tempo da questão, como explicado na seção acima. Portanto, precisamos de uma solução que otimize o tempo gasto no algoritmo. Já a complexidade de espaço é $\mathcal{O}(n)$; por ser linear, é aceitável para esse problema.

```
1 #include <bits/stdc++.h>
2 #define ll long long
3
4 using namespace std;
5
6 int main()
7 {
8     int size;
9     cin >> size;
10    vector<ll> array (size), LIS (size,1);
11    int longest = (size > 0);
12
13    for (int i = 0; i < size; i++)
14    {
15        cin >> array[i];
16
17        for (int j = 0; j < i; j++)
18        {
19            if (array[j] < array[i])
20            {
21                LIS[i] = max(LIS[i], LIS[j]+1);
22
23                if (longest < LIS[i])
24                    longest = LIS[i];
25            }
26        }
27    }
28
29    cout << longest << endl;
30    return 0;
31 }
```

4 Soluções Corretas e Otimizadas

4.1 Descrição do Algoritmo Principal

A solução otimizada do problema seguirá a seguinte ideia ao invés de avaliar para cada subarray o tamanho da maior subsequência crescente contida nele, construiremos um algoritmo que armazena, para cada posição da subsequência de interesse o valor do menor elemento que pode finalizar a subsequência nesta posição (essa descrição ficará mais clara quando discutirmos o algoritmo e a solução utilizados).

A solução descrita continuará sendo uma abordagem do problema a partir do paradigma de programação dinâmica, uma vez que o algoritmo utilizado, operará dividindo o problema em problemas menores, cuja solução é mais simples, e memorizando os resultados obtidos para que, caso eles sejam novamente necessários, eles já foram calculados e armazenados, caracterizando-se assim como uma solução utilizando DP.

Com intuito de implementar a solução descrita acima, vamos adaptar um algoritmo de *sorting* (um algoritmo que ordena / classifica / separa os elementos de um conjunto segundo algum critério especificado) inspirado no jogo de cartas de baralho paciência. O jogo funciona da seguinte forma: inicialmente as cartas são embaralhadas e dispostas em pilhas de tal forma que cada pilha da esquerda para direita tem sempre uma carta virada para cima e uma carta virada para baixo a mais que a pilha anterior (começando na 1ª pilha com zero cartas viradas para baixo) e o restante das cartas serve como monte para compra. O objetivo do jogo é formar 4 pilhas (uma para cada naipe) de tal forma que cada uma delas contenha uma sequência decrescente começando no Rei e terminando no Ás, ou seja a sequência {K, Q, J, 10, 9, 8, 7, 6, 5, 4, 3, 2, A}.

O algoritmo proposto inspirado no jogo, conhecido como *Patience Sorting*, funcionará, da seguinte forma: inicializarmos uma estrutura auxiliar que permitirá armazenar as diversas "pilhas" que processaremos do *container* original. Depois disso processaremos cada elemento i do *container* original e o inseriremos na estrutura auxiliar seguindo um dos critérios listados abaixo:

- Caso o elemento analisado seja menor que um elemento que encontra-se no topo da posição j da estrutura auxiliar, o elemento deve ser inserido na pilha do elemento j e o elemento será o novo topo da pilha j , em que a pilha j representa a primeira pilha na qual o elemento é menor ou igual ao elemento do topo.
- Caso o elemento analisado seja maior que todos os elementos que estão nos topos das pilhas contidas na estrutura auxiliar então deve-se criar uma nova pilha na estrutura que terá como topo este elemento.

Note que a inserção dos elementos seguindo esse padrão garante que dois elementos inseridos na mesma pilha não podem integrar simultaneamente a mesma subsequência crescente, isso porque sendo o elemento do topo menor que um elemento da base só haveria uma subsequência crescente nesse caso se o topo ocorresse no array antes da base, o que nos critérios definidos é impossível, então a sequência de interesse, obrigatoriamente, deve ocorrer entre as

pilhas sendo assim concluímos que o tamanho máximo para a maior subsequência crescente deve ser menor ou igual à quantidade de pilhas; porém como os critérios definidos explicitam que só podemos criar novas pilhas quando ocorre que o elemento é maior que todos os topos das pilhas até então criadas na estrutura, então deve existir algum caminho ótimo que conecte exatamente um elemento por pilha e passe por todas as pilhas, uma vez que para criarmos uma nova pilha foi preciso que um elemento fosse maior que todos os topos das pilhas atuais e nesse cenário conseguiríamos conceber tal caminho. Portanto o tamanho da maior subsequência crescente será dada pela quantidade de pilhas criadas, isto é, pelo tamanho da estrutura auxiliar. Para ilustrar o funcionamento do algoritmo descrito, considere o seguinte diagrama (figura (1)) que representa o funcionamento do algoritmo para o caso de teste apresentado no problema:

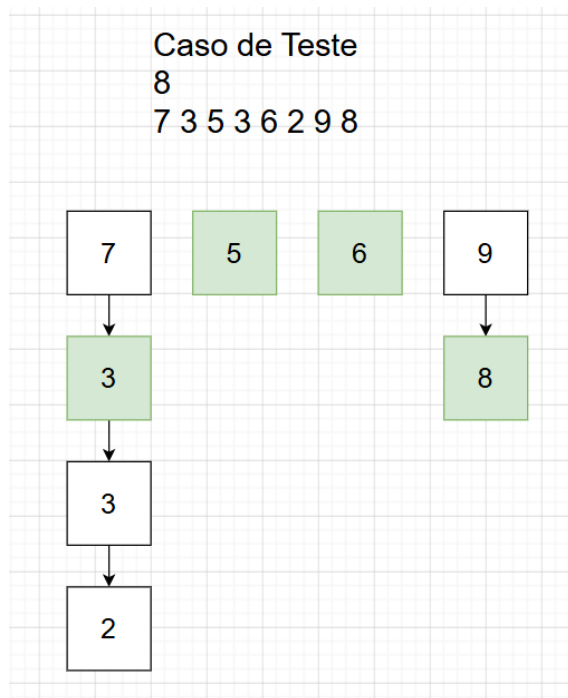


Figura 1: Estrutura auxiliar e das pilhas criadas durante a execução do algoritmo

A figura (1) expõe como a estrutura auxiliar proposta seria alterada ao percorrermos os elementos do array original, note que o algoritmo proposto precisa apenas dos topos das pilhas para realizar as comparações sendo assim a estrutura de dados que será utilizada nessa tarefa é simplesmente um **vector** onde para criar pilhas novas realizaremos o método `push_back(elemento)` por meio do qual inseriremos um elemento ao final do *container* e para atualizar o valor do topo **sobrescreve-se** o valor no *vector* para alterá-lo.

Além disso percebe-se que a estrutura auxiliar descrita não armazena a maior subsequência crescente existente no array, ao final da execução do algoritmo a estrutura auxiliar terá a seguinte composição: 2, 5, 6, 8 que não representa a subsequência de interesse (uma das subsequências de interesse existentes está marcada pela cor verde no diagrama), porém uma

vez que o problema solicita apenas o tamanho da maior subsequência e não os elementos que a compõe, então a solução descrita é suficiente para resolver o problema.

Além do algoritmo principal descrito e provado acima é preciso escolher também um algoritmo de busca pois para cada elemento do array será preciso determinar a posição que le deve se inserido ou se é preciso criar uma nova pilha. Haja visto que o array está sempre ordenado, pelos critérios definidos de inserção e atualização dos elementos, então uma alternativa eficiente é realizarmos uma busca binária para realizar essas consultas, utilizando a função em linguagem cpp que implementa a busca binária e retorna um iterador para a posição do elemento que é maior ou igual que o elemento alvo: `lower_bound(arr.begin(), arr.end(), elemento)`.

4.2 Implementação em Código

O código que implementa a solução ótima descrita na seção anterior é o seguinte:

```
1 #include <bits/stdc++.h>
2
3 #define ll long long
4 #define vi vector<int>
5 #define vll vector<ll>
6 #define endl '\n'
7 #define FASTIO ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
8
9 using namespace std;
10
11 int main()
12 {
13     FASTIO
14     int size; cin >> size;
15     vll array(size), aux;
16
17     for(int i = 0; i < size; i++)
18     {
19         cin >> array[i];
20         auto pos = lower_bound(aux.begin(), aux.end(), array[i]);
21         if(pos == aux.end())
22             aux.push_back(array[i]);
23         else
24             aux[pos-aux.begin()] = array[i];
25     }
26
27     cout << aux.size() << endl;
28
29     return 0;
30 }
```

4.3 Análise de Complexidade

Para simplificar a análise do algoritmo considere que a letra n diz respeito ao tamanho do array, referenciando o código proposto $n = size$.

- Tempo $\mathcal{O}(n \log n)$.
- Espaço: $\mathcal{O}(n)$.

5 Considerações Finais

O problema proposto embora pareça, num primeiro momento bem simples, se revelou, em uma análise mais detalhada, como um problema relativamente elaborado, que precisou ser resolvido utilizando algoritmos específicos que conseguissem performar dentro das limitações impostas, a questão foi um exercício interessante de estudo e análise de algoritmos mais específicos e permitiu aprofundar nossos conhecimentos em programação dinâmica e algoritmos de sorting.

A principal dificuldade do problema foi lidar com a limitação de tempo para processamento do array, uma vez que a solução trivial é bastante intuitiva e simples, mas não satisfaz o limite de tempo. A solução necessária se mostra mais difícil de entender e provar, mas em código é mais curta e objetiva. Uma extensão desse problema é exigir que a saída não seja apenas o tamanho da LIS, mas sim os elementos que compõem a subsequência, o que talvez seja obtido adaptando o algoritmo proposto. Além disso, outro problema relacionado poderia ser a determinação da maior subsequência decrescente (LDS) existente no array.